

Chapter 7

Fast-Fourier Transform (FFT) Algorithm

Contents

- 7.1 The DFT Review
- 7.2 Decimation-in-Time FFT
- 7.3 Operation Count, Bit Reversal, and In-Place Computation
- 7.4 Decimation-in-Frequency FFT
- 7.5 IDFT Computation using an FFT
- 7.6 Applications of FFT Algorithms

7.1 The DFT and DFT Matrix

- DFT of an N -point sequence $x[n]$, $n = 0, 1, 2, \dots, N - 1$ is defined as

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi k}{N}n}, \quad k = 0, 1, 2, \dots, N - 1.$$

- An N -point sequence yields an N -point transform.
- $X[k]$ can be expressed as an *inner product*. Notation: $W_N = e^{-j\frac{2\pi}{N}}$. Hence,

$$X[k] = \begin{bmatrix} 1 & W_N^k & W_N^{2k} & \dots & W_N^{(N-1)k} \end{bmatrix} \begin{bmatrix} x[0] \\ x[1] \\ \vdots \\ x[N-1] \end{bmatrix}.$$

7.1 The DFT and DFT Matrix

- By varying k from 0 to $N - 1$ and combining the N inner products, we get the following:

$$\mathbf{X} = \mathbf{W} \mathbf{x}.$$

- $\mathbf{W}$ is an $N \times N$ matrix, called the “DFT Matrix”.

$$\mathbf{W} = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & W_N & W_N^2 & \cdots & W_N^{N-1} \\ 1 & W_N^2 & W_N^4 & \cdots & W_N^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & W_N^{N-1} & W_N^{2(N-1)} & \cdots & W_N^{(N-1)(N-1)} \end{bmatrix}_{N \times N}.$$

7.1 How Many Complex Multiplications Are Required?

- Each inner product requires N complex multiplications
 - There are N inner products
- Hence we require N^2 multiplications
- However, the first row and first column are all 1s, and *should not be counted as multiplications*
 - There are $2N - 1$ such instances
- Hence, the number of complex multiplications is $N^2 - 2N + 1$, i.e., $(N - 1)^2$

7.1 How Many Complex Additions Are Required?

- Each inner product requires $N - 1$ complex additions
 - There are N inner products
- Hence we require $N(N - 1)$ complex additions

7.1 Total Operation Count

- No. of complex multiplications: $(N - 1)^2$
- No. of complex additions: $N(N - 1)$
- The operation count for multiplications and additions assumes that W_N^k has been computed offline and is available in memory
- We will assume that all the required W_N^k have been pre-computed and are available

7.1 Operation Count Makes DFT Impractical

- For large N ,

$$(N - 1)^2 \approx N^2$$

$$N(N - 1) \approx N^2$$

- Hence both multiplications and additions are $O(N^2)$
- If $N = 10^3$, then $O(N^2) = 10^6$, i.e., a million!
- This makes the straightforward method slow and impractical even for a moderately long sequence

How can we compute the DFT much faster than the straightforward $O(N^2)$ method?

7.1 The Divide and Conquer Approach

- Suppose N is even and we split the sequence into two halves.
 - Each sequence has $\frac{N}{2}$ points
- Suppose we compute the $\frac{N}{2}$ point DFT of each sequence
 - Multiplications: $2 \times \left(\frac{N}{2}\right)^2 = \frac{N^2}{2}$
- Suppose we are able to combine the individual DFT results to get the originally required DFT
 - Some computational overhead will be consumed to combine the two results
- If $\frac{N^2}{2} + \text{overhead} < N^2$, then this approach will reduce the operation count

7.1 The Divide and Conquer Approach

Let $N = 8$

- Straightforward implementation requires, *approximately*, 64 multiplications
- The “divide and conquer” approach requires, *approximately*,

$$2 \times \left(\frac{8}{2}\right)^2 + \text{overhead, i.e., } 32 + \text{overhead multiplications}$$

- Questions:
 - Can the two DFTs be combined to get the original DFT?
 - If so, how? What is the overhead involved?
 - Will $32 + \text{overhead}$ be less than 64?

7.2 The Decimation in Time (DIT) Algorithm

- From $\{x[n]\}$ form two sequences as follows:

$$\{g[n]\} = \{x[2n]\} \qquad \{h[n]\} = \{x[2n+1]\}$$

- $\{g[n]\}$ contains the even-indexed samples, while $\{h[n]\}$ contains the odd-indexed samples
- The DFT of $\{x[n]\}$ is

$$\begin{aligned} X[k] &= \sum_{n=0}^{N-1} x[n] W_N^{nk} \\ &= \sum_{r=0}^{\frac{N}{2}-1} x[2r] W_N^{(2r)k} + \sum_{r=0}^{\frac{N}{2}-1} x[2r+1] W_N^{(2r+1)k} \\ &= \sum_{r=0}^{\frac{N}{2}-1} g[r] W_N^{(2r)k} + W_N^k \sum_{r=0}^{\frac{N}{2}-1} h[r] W_N^{(2r)k} \end{aligned}$$

7.2 The Decimation in Time (DIT) Algorithm

- But,

$$W_N^{2rk} = e^{-j\frac{2\pi}{N}(2rk)} = e^{-j\frac{2\pi}{N/2}(rk)} = W_{N/2}^{rk}$$

and hence

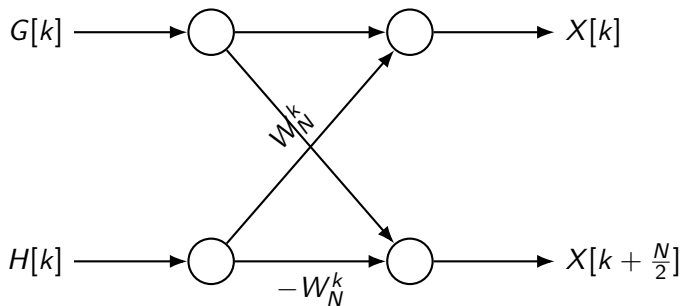
$$\begin{aligned} X[k] &= \sum_{r=0}^{\frac{N}{2}-1} g[r] W_{N/2}^{rk} + W_N^k \sum_{r=0}^{\frac{N}{2}-1} h[r] W_{N/2}^{rk} \\ &= G[k] + W_N^k H[k], \quad k = 0, 1, \dots, N-1 \end{aligned}$$

- $\{G[k]\}$ and $\{H[k]\}$ are $\frac{N}{2}$ point DFTs
- The overhead for combining the two $N/2$ point DFTs is the multiplicative factor W_N^k for $k = 0, 1, \dots, N-1$
 - W_N^k is called “twiddle factor”

7.2 The Decimation in Time (DIT) Algorithm

- The $N/2$ point DFTs $\{G[k]\}$ and $\{H[k]\}$ are periodic with period $N/2$
 - $G[k + \frac{N}{2}] = G[k]$
 - $H[k + \frac{N}{2}] = H[k]$
- $W_N^{k+\frac{N}{2}} = -W_N^k$
- Hence, if $X[k] = G[k] + W_N^k H[k]$, then $X[k + \frac{N}{2}] = G[k] - W_N^k H[k]$
 - $W_N^k H[k]$ needs to be computed only once for $k = 0$ to $\frac{N}{2} - 1$
- Thus, the multiplication overhead due to the twiddle factors is only $\frac{N}{2}$

7.2 Butterfly Diagram



$$X[k] = G[k] + W_N^k H[k], \quad X[k + \frac{N}{2}] = G[k] - W_N^k H[k].$$

7.2 The Decimation in Time (DIT) Algorithm

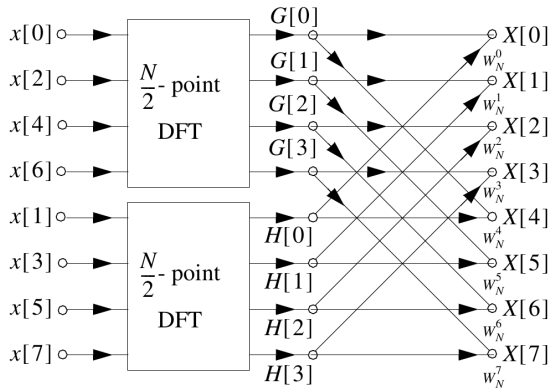


Figure: Flowgraph of Decimation in Time algorithm for $N = 8$

7.2 “Divide and Conquer” Results in Savings

- For $N = 8$, the straightforward approach requires, *approximately*, 64 multiplications
- The “Divide and Conquer” approach, after the first stage, requires $32 + 4 = 36$ multiplications
- Thus, this approach clearly reduces the number of additions and multiplications required.

7.2 Reusing the “Divide and Conquer” Strategy

- The same idea can be applied for calculating the $\frac{N}{2}$ point DFT of the sequences $\{g[r]\}$ and $\{h[r]\}$
 - Computational savings can be obtained by dividing $\{g[r]\}$ and $\{h[r]\}$ into *their* odd- and even-indexed halves
- This idea can be applied recursively $\log_2 N$ times if N is a power of 2
- If $N = 2^\gamma$, then the final stage sequences are all of length 2
- For a 2-point sequence $\{p[0], p[1]\}$, the DFT coefficients are

$$P[0] = p[0] + p[1] \quad P[1] = p[0] - p[1]$$

7.2 DIT Flowgraph for $N = 8$

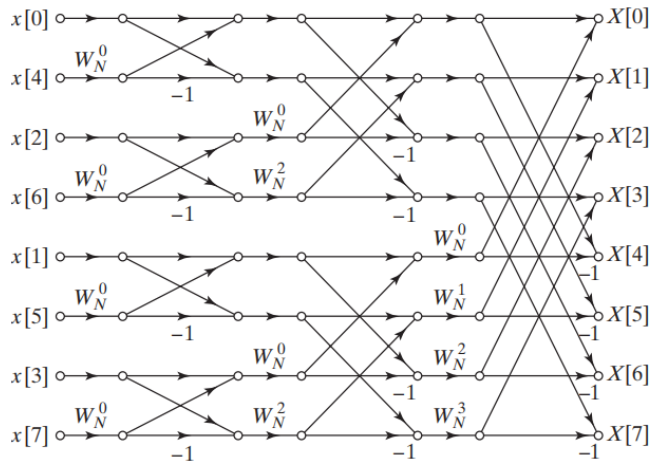


Figure: Flowgraph of Decimation in Time algorithm for $N = 8$

7.3 Overall Operation Count

- The direct method requires N^2 multiplications
- After the first split,

$$N^2 \longrightarrow 2 \left(\frac{N}{2} \right)^2 + \frac{N}{2}$$

- $\frac{N}{2}$ is due to the *twiddle factors*
- After the second split,

$$\left(\frac{N}{2} \right)^2 \longrightarrow 2 \left(\frac{N}{4} \right)^2 + \frac{N}{4}$$

- Hence,

$$N^2 \longrightarrow 2 \left(\frac{N}{2} \right)^2 + \underbrace{\frac{N}{2}}_{\text{first stage}} \longrightarrow 4 \left(\frac{N}{4} \right)^2 + \underbrace{\frac{N}{2} + \frac{N}{2}}_{\text{second stage}}$$

- Generalizing, if there are $\log_2 N$ stages, the number of multiplications needed will be, *approximately*, $\frac{N}{2} \log_2 N$

7.3 Overall Operation Count

- If $W_N^{k+\frac{N}{2}} = -W_N^k$ is not considered, the overhead count will be N and not $\frac{N}{2}$
- In this case,

$$N^2 \longrightarrow 2\left(\frac{N}{2}\right)^2 + N \longrightarrow 4\left(\frac{N}{4}\right)^2 + N + N$$

- Hence the overall multiplication count will be $N \log_2 N$
- For $N = 1024$

$$N^2 = 1,048,576$$

$$N \log_2 N = 10,240$$

Savings of two orders of magnitude!

7.3 Input Sequence Order

- Recall that, for $N = 8$, the first split requires the data to be arranged as follows:

$$x[0], x[2], x[4], x[6], \quad x[1], x[3], x[5], x[7]$$

- In the second and final split, the data appear in the following order:

$$x[0], x[4], \quad x[2], x[6], \quad x[1], x[5], \quad x[3], x[7]$$

- The final order is said to be in “bit reversed” form:

Original	Binary Form	Reversed Form	Final
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7

7.3 An Algorithm For Sequence Reversal

- Consider the card sequence 7, 8, 9, 10, J, Q, K, A
- First, reverse pairwise:
 - 8, 7, 10, 9, Q, J, A, K
- Then swap the adjacent pairs:
 - 10, 9, 8, 7, A, K, Q, J
- Finally, swap the two groups of 4 (each group is half the original size):
 - A, K, Q, J, 10, 9, 8, 7 *Done!*

7.3 In-Place Computation

Notation:

$$X^{(0)}[k] = x[k], \quad X^{(\log_2 N)}[k] = X[k].$$

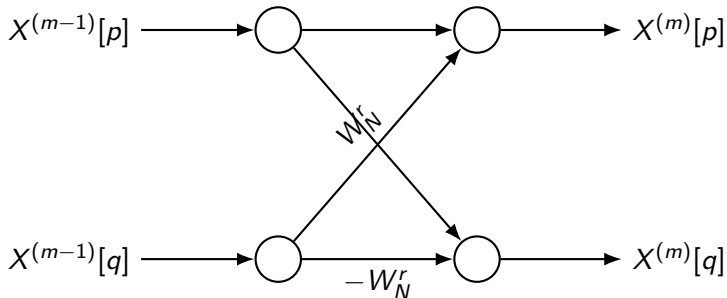
For the m -th stage butterfly:

- Input: $X^{(m-1)}[p]$, $X^{(m-1)}[q]$
- Output: $X^{(m)}[p]$, $X^{(m)}[q]$

The corresponding equations are

$$\begin{aligned} X^{(m)}[p] &= X^{(m-1)}[p] + W_N^r X^{(m-1)}[q], \\ X^{(m)}[q] &= X^{(m-1)}[p] - W_N^r X^{(m-1)}[q]. \end{aligned}$$

7.3 In-Place Computation



- $X^{(m-1)}[p]$ and $X^{(m-1)}[q]$ are needed for computing $X^{(m)}[p]$ and $X^{(m)}[q]$.
- They are not needed for any other pair, hence they can be overwritten:

$$X^{(m)}[p] \leftarrow X^{(m-1)}[p], \quad X^{(m)}[q] \leftarrow X^{(m-1)}[q].$$

- This is called *in-place computation*.

7.4 Decimation-in-Frequency FFT Algorithms

- With the *decimation-in-frequency* (DIF) algorithms, decomposition is performed by decimating the output sequence $X[k]$.
- A DIF algorithm is developed by viewing $X[k]$ as consisting of even- and odd-indexed frequency samples.
- The starting point is the N -point DFT:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad W_N = e^{-j2\pi/N}.$$

- The trick is to split the time index into the first and second halves, then separate the resulting expression according to whether k is even or odd.

7.4 DIF Derivation: Split the DFT Sum

Begin by writing $X[k]$ as two sums:

$$X[k] = \sum_{n=0}^{N/2-1} x[n] W_N^{kn} + \sum_{n=N/2}^{N-1} x[n] W_N^{kn}.$$

Let $r = n - N/2$ in the second sum. Then

$$X[k] = \sum_{n=0}^{N/2-1} x[n] W_N^{kn} + \sum_{r=0}^{N/2-1} x[r + N/2] W_N^{k(r+N/2)}.$$

Since

$$W_N^{kN/2} = \left(e^{-j2\pi/N} \right)^{kN/2} = e^{-j\pi k} = (-1)^k,$$

we obtain

$$X[k] = \sum_{n=0}^{N/2-1} [x[n] + (-1)^k x[n + N/2]] W_N^{nk}.$$

7.4 DIF Derivation: Even Frequency Samples

Consider the even-indexed frequency samples:

$$k = 2r, \quad r = 0, 1, \dots, N/2 - 1.$$

Substituting $k = 2r$ gives

$$X[2r] = \sum_{n=0}^{N/2-1} \underbrace{[x[n] + x[n + N/2]]}_{g[n]} W_N^{2nr}.$$

Using $W_N^2 = W_{N/2}$,

$$X[2r] = \sum_{n=0}^{N/2-1} g[n] W_{N/2}^{nr}.$$

Meaning: The even-indexed DFT output samples $X[0], X[2], \dots, X[N-2]$ are obtained from an $N/2$ -point DFT of

$$g[n] = x[n] + x[n + N/2].$$

7.4 DIF Derivation: Odd Frequency Samples

Consider the odd-indexed frequency samples:

$$k = 2r + 1, \quad r = 0, 1, \dots, N/2 - 1.$$

Substituting $k = 2r + 1$ gives

$$X[2r + 1] = \sum_{n=0}^{N/2-1} \underbrace{[x[n] - x[n + N/2]]}_{h[n]} W_N^n W_{N/2}^{nr}.$$

Meaning: The odd-indexed DFT output samples $X[1], X[3], \dots, X[N - 1]$ are obtained from an $N/2$ -point DFT of

$$h[n] W_N^n, \quad h[n] = x[n] - x[n + N/2].$$

7.4 The Two $N/2$ -Point DFTs in DIF

- The sequences $g[n]$ and $h[n]$ are both $N/2$ -point sequences formed from the input by simple combining algebra:

$$g[n] = x[n] + x[n + N/2], \quad h[n] = x[n] - x[n + N/2].$$

- The output sequence $X[k]$ is obtained by computing two $N/2$ -point DFTs:

$$X[2r] = \text{DFT}_{N/2}\{g[n]\}[r],$$

$$X[2r + 1] = \text{DFT}_{N/2}\{h[n]W_N^n\}[r].$$

- The even-indexed frequency samples appear together, and the odd-indexed frequency samples appear together.

7.4 One Stage of DIF Decomposition

- For an $N = 8$ DFT, one stage of DIF decomposition forms $g[0], \dots, g[3]$ and $h[0], \dots, h[3]$.
- Two $N/2$ -point DFTs then produce the even and odd frequency samples.

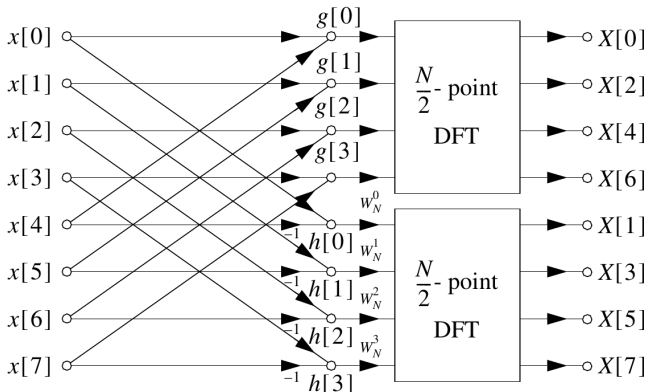
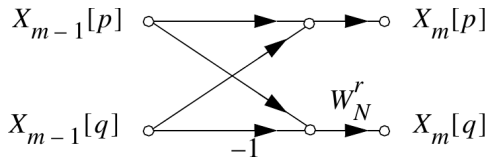


Figure: Flowgraph of Decimation in Frequency algorithm for $N = 8$

7.4 Continuing the DIF Decomposition

- By further decomposing the sequences $g[n]$ and $h[n]$ into even and odd parts, the DFT is reduced to $N/4$ -point transforms.
- The same input-side combining algebra is applied recursively.
- After $\log_2 N$ decompositions, the basic computational element is again a 2-point DFT.



Simplified radix-2 DIF butterfly.

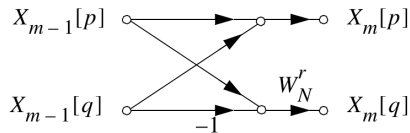
7.4 Simplified DIF Butterfly

For a butterfly pair entering stage m , the DIF butterfly performs

$$X^{(m)}[p] = X^{(m-1)}[p] + X^{(m-1)}[q],$$

$$X^{(m)}[q] = \left(X^{(m-1)}[p] - X^{(m-1)}[q] \right) W_N^r.$$

- Compared with the DIT butterfly, the twiddle factor multiplication appears after the subtraction branch.
- This is the computation shown in the simplified butterfly diagram.



7.4 DIF Flowgraph for $N = 8$

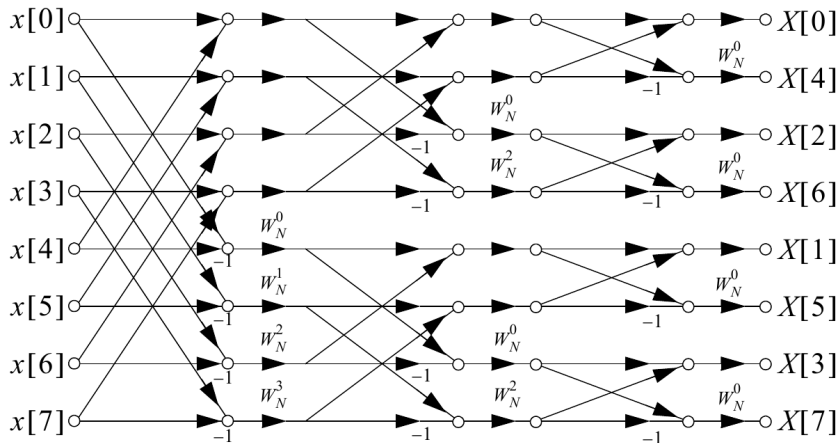


Figure: Flowgraph of Decimation in Frequency algorithm for $N = 8$

7.4 DIF Computation Count and Observations

- The total computation count when using the simplified DIF butterfly is once again

$$\frac{N}{2} \log_2 N \quad \text{complex multiplications}$$

and

$$N \log_2 N \quad \text{complex additions.}$$

- The decimation-in-time (DIT) and decimation-in-frequency (DIF) flowgraphs are related by *network transposition*.
- DIF FFT observations:
 - the input is in normal order;
 - the output is in bit-reversed order, the opposite of the DIT FFT;
 - the butterfly structure allows in-place computations.

7.5 IDFT Computation using an FFT

To compute the IDFT using one of the FFT algorithms just described is quite simple.

- The computation is identical to the DFT except for a scale factor $1/N$.
- The twiddle factors must be conjugated. With

$$W_N = e^{-j2\pi/N},$$

the DFT uses W_N^{kn} while the IDFT uses $W_N^{-kn} = (W_N^{kn})^*$.

DFT and IDFT pair:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-kn}.$$

7.5 IDFT as a Conjugated DFT

Another way of looking at the IDFT is to rewrite it in terms of an ordinary DFT.

$$\begin{aligned}x[n] &= \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-kn} \\&= \frac{1}{N} \left[\sum_{k=0}^{N-1} X^*[k] W_N^{kn} \right]^*.\end{aligned}$$

Therefore,

$$x[n] = \frac{1}{N} [\text{DFT} \{X^*[k]\} [n]]^*$$

- Conjugate the input spectrum $X[k]$.
- Run the usual FFT.
- Conjugate the FFT output and scale by $1/N$.

7.5 Practical IDFT Procedure using an FFT

A practical FFT-based IDFT can be summarized as follows:

1. Start with the DFT samples $X[k]$, $0 \leq k \leq N - 1$.
2. Form the conjugated sequence $X^*[k]$.
3. Compute

$$U[n] = \text{FFT}\{X^*[k]\}.$$

4. Recover the IDFT samples by

$$x[n] = \frac{1}{N} U^*[n], \quad 0 \leq n \leq N - 1.$$

The same FFT engine can compute both the DFT and the IDFT. The only extra operations are conjugations and the final $1/N$ scaling.

7.6 Applications of FFT Algorithms

The FFT is by definition designed to operate on complex input data.

- The input may be real, but the weights W_N^k are generally complex.
- The array used for in-place computations must always be complex.
- Suppose $x_1[n]$ and $x_2[n]$ are two real N -point sequences.
- Define a complex sequence

$$x[n] = x_1[n] + jx_2[n], \quad 0 \leq n \leq N - 1.$$

Since the DFT is a linear operator,

$$X[k] = X_1[k] + jX_2[k],$$

where $X_1[k]$ and $X_2[k]$ are, of course, complex quantities.

7.6 Separating the Two Real DFTs

To extract $X_1[k]$ and $X_2[k]$ from $X[k]$, first note that

$$x_1[n] = \frac{1}{2} \{x[n] + x^*[n]\}, \quad x_2[n] = \frac{1}{2j} \{x[n] - x^*[n]\}.$$

Taking the DFT of both sides gives

$$X_1[k] = \frac{1}{2} [\text{DFT}\{x[n]\} + \text{DFT}\{x^*[n]\}],$$

$$X_2[k] = \frac{1}{2j} [\text{DFT}\{x[n]\} - \text{DFT}\{x^*[n]\}].$$

Using the DFT property

$$x^*[n] \xleftrightarrow{\text{DFT}} X^*[N - k],$$

where the index is understood modulo N , we obtain the formulas on the next slide.

7.6 Two Real DFTs from One Complex FFT

For $0 \leq k \leq N - 1$,

$$X_1[k] = \frac{1}{2} \{X[k] + X^*[N - k]\}$$

$$X_2[k] = \frac{1}{2j} \{X[k] - X^*[N - k]\}$$

with all indices interpreted modulo N .

Algorithm:

1. Pack the two real sequences into $x[n] = x_1[n] + jx_2[n]$.
2. Compute one N -point FFT to get $X[k]$.
3. Use the two symmetry formulas above to recover $X_1[k]$ and $X_2[k]$.